EPT $\rightarrow$ AI-era mathematics: project handoff

Current state and prioritized next moves

August 12, 2026

This document is a self-contained briefing for whoever (human or model) picks up the project next. It records what has been established, where every result lives in the repository, what infrastructure assumptions have *changed* (disk is no longer scarce; cloud instances are available), and a prioritized experiment queue — led by the full-unfolding subterm experiment, which tests whether the current results depend on treating the named-lemma library as fixed.

1 Project in one paragraph

Viteri & DeDeo (*Cognition* 2022) modeled mathematical proofs as dependency networks, found heavy-tailed premise reuse ($\alpha \approx 2$ power laws), and showed that an asymmetric Ising (Glauber) belief dynamic on these networks exhibits epistemic phase transitions (EPTs): abrupt collective transitions to certainty, with modular "firewalls" containing local doubt. This project asks whether those structural signatures survive in AI-era mathematics, using Lean 4 megaprojects (PFR, FLT, Sphere Packing, equational_theories) that mix human-written, AI-assisted, and pure-search-generated proofs, plus Mizar human-vs-ATP paired proofs and AlphaProof-era corpora. Everything is extracted at up to four grains: blueprint (informal LaTeX `\uses`), textual Lean (regex), kernel (compiled environment, `Expr.getUsedConstants`), and subterm (hash-consed proof-term DAGs, the 2022 paper's original Coq grain, reimplemented in Lean from the 2019 `ManipulateProofTrees` algorithm).

2 State of the evidence (post-audit)

All numbers below survived a user-prompted audit round that fixed four extractor bugs (Unicode names, the Lean `public` modifier, string-literal phantoms, short-name capture) and retracted two early claims ("ER blurs the transition" — it does not, ER is sharp at $\varepsilon \approx 0.44$; sphere sorry-clamping -0.007 was seed luck, honest value -0.07 ± 0.10). Sources: `README.md` (findings list), `paper/main.tex` (11-page draft), `results/*.json`.

1. **Two-grain law.** Within the same projects, informal blueprints show the "Wiles signature" ($\alpha \approx 3.3$–4.6) while their Lean formalizations sit at $\alpha \approx 2$. Kernel-grain ground truth: PFR 2.46, FLT 2.99, Sphere 2.49, Gauss (math-inc) 2.56, ET 2.88.
2. **Heavy tails are real but "power law" is corpus-specific.** Semiparametric bootstrap GOF: power law not rejected for PFR/FLT/Sphere/math-inc ($p = .15$–$.85$), rejected for ET ($p < .01$). Vuong vs. lognormal is decisive only for ET. Honest phrasing: "approximately power-law heavy tails."

3. **The machine stratum is one-way and unreused.** ET generated certificates: 200,614 human→machine edges vs. 37 back; 0.15% of certificates ever reused.

4. **The dust is task-sorted, not agent-sorted.** Human proofs of *ordinary* ET implications are exactly as flat as machine ones (median construction-specific support 1 for both); only deep instances (counterexample `Facts`: median 33 supporting constructions) show the tower economy, and machines never reach those. Refined claim: *search operates exactly where dust suffices.*

5. **The general-task control agrees.** Mizar: 1,274 matched theorems, human premises α 2.36/Gini 0.740 vs. ATP α 3.18/Gini 0.567 (0.669 size-matched); only 29% of ATP premises lie in the human dependency set. Caveat: Mizar40 premise selection was kNN-trained on human proofs.

6. **Mechanism decomposition of the EPT.** The belief curve is a degree-sequence phenomenon (configuration model reproduces it); the firewall ΔL_1 is a modularity phenomenon. There is no susceptibility peak — the real transition-like structure is *bistability* (hysteresis at every $\varepsilon \leq 0.46$). Causal arm: synthetic $\alpha \in [1.8, 4.0]$ sweep shows onset ε_c tracks $\langle k^2 \rangle / \langle k \rangle$ ($0.43 \to 0.24$); heavy tails buy *early onset, staged hub-first ordering, and core robustness*, not sharpness.

7. **Asymmetric couplings are a plateau.** EPT exists at every β_{dep}:β_{imp} ratio from 1:4 to 4:1. Refutation propagates only through fragile 1–2-consequence links and only when $\beta_{\mathrm{imp}} > \beta_{\mathrm{dep}}$; reuse hubs are refutation-proof (Lakatos, quantified).

8. **Correspondence (tracking) degrades with machine mass.** Informal edges realized as formal paths, kernel grain: PFR 86% > FLT 79% > Sphere 66% > ET 43% (random-pair base rates 2–11%).

9. **Below the naming grain, machine proofs are unfactored, not structureless.** Subterm analysis of all 47,940 ET proof terms: size-matched internal compression is identical (human $10^{0.88}$ vs. machine $10^{0.87}$) and machine `maxref` is *higher* (39 vs. 24). But the character differs: human towers achieve diverse deep compression (`Facts` median 10^2; one term's $10^{10.8}$-node tree collapses to a 124,638-node DAG), machine sharing is concentrated repetition that never gets factored into named declarations. *What search lacks is naming, not repetition.*

10. **Unfinished proofs.** Blueprints do cover unproved parts (FLT 39% proof-done, Sphere 35%, PFR 86%); `sorryAx` taint is small (0.7–9.2%). FLT sorries are frontier targets (clamping them: downstream belief -0.25 ± 0.09); Sphere sorries are scaffolding shielded by redundant paths (-0.07 ± 0.10).

3 Repository map

Repo: `https://github.com/scottviteri/ept-ai-analysis` (public; GitHub Pages serves `docs/index.html` at `https://scottviteri.com/ept-ai-analysis/`, an interactive report with a live Glauber animation on the PFR kernel network).

Question	**Script**	**Results**
CSN degree stats, α fits	`analysis/analyze.py`	`results/results_all.`
Textual Lean extraction	`analysis/extract_lean.py`	`graphs/`
Kernel extraction (per-project)	`analysis/extract_deps{,_legacy}.lean`	`results/*_kernel_dep`
orchestration	`analysis/kernel_pipeline.sh`	`results/*_kernel_val`
Blueprint extraction	`analysis/extract_blueprint.py`	`graphs/`
Tracking fidelity	`analysis/track.py`	`results/tracking_res`
Null models + asymmetry	`analysis/null_models.py`	`results/null_models{`
2D ($\beta_{\mathrm{dep}}, \beta_{\mathrm{imp}}$) plane	`analysis/asymmetric_2d.py`	`results/asymmetric_2`
Transition existence (GOF, χ, hysteresis, α sweep)	`analysis/transition_tests.py`	`results/transition_t`
Robustness (Vuong, jackknife, ...)	`analysis/robustness.py`	`results/robustness_s`
ET depth split (dust confound)	`analysis/et_depth_split.py`	`results/et_depth_spl`
Mizar human vs. ATP	`analysis/mizar_atp_compare.py`	`results/mizar_atp_co`
Sorry taint + clamping	`analysis/sorry_{analysis,ising}.py`	`results/*_sorry_*.js`
Subterm hash-consing (Lean)	`analysis/subterm_stats.lean`	`results/et_subterm_s`
Subterm strata comparison	`analysis/subterm_analysis.py`	`results/et_subterm_s`
Figures	`analysis/gen_figures.py`	`figs/`
Paper draft (11pp)	`paper/main.tex`	`paper/main.pdf`

What is saved vs. what needs a rebuild. The `results/*_kernel_deps.jsonl` files store *constant-name* dependencies only — enough for every graph-level analysis without any Lean toolchain. Proof-term *Exprs* are not saved (they are the multi-GB `.olean` payload); any new subterm-grain question needs the compiled environment. ET's per-declaration subterm statistics are saved (`et_subterm_stats.jsonl.gz`), so re-analysis at that grain needs no rebuild; new per-term quantities (e.g. unfolding, §4.1) do.

4 Next moves, prioritized

4.1 P1: Full unfolding — does dissolving the naming grain change the results?

Question. Every current subterm result holds the library factoring fixed: each declaration's proof term is analyzed with cited lemmas left as opaque `Expr.const` leaves. The 2019/2022 pipeline instead *substituted* definitions in (to axioms) and hash-consed across lemma boundaries. Does the human-vs-machine comparison change when names are dissolved — i.e. when we measure the proof's *total* structure down to term-level constants rather than its structure *given* the named library?

Why it matters. The headline claim of §2.9 is "what search lacks is naming, not repetition." If, after unfolding, human and machine terms become statistically indistinguishable, that claim sharpens: the entire human–machine gap lives in the naming economy. If human terms still show more diverse cross-lemma sharing after names are dissolved, the gap is deeper than naming and the claim must weaken. Either outcome is informative, and this is the main untested degree of freedom left in the subterm analysis. It is also the exact bridge to the 2022 baseline, whose Coq networks were built from fully expanded terms.

Implementation sketch (extend `analysis/subterm_stats.lean`; no serialization, so no exponential blowup — this is the same trick that made the current version cheap):

1. Maintain a *global* memo table keyed by `(Name, List Level)`: a constant's unfolded statistics

depend on its universe instantiation, so key on both, and instantiate with `ConstantInfo.instantiateValueLe` before walking. Measure the number of distinct instantiations per constant first — if it explodes, fall back to keying on `Name` alone and note the approximation.

2. During the walk of a term, when hitting `Expr.const n ls` where `n` is *project-internal* and has a value, do not treat it as a leaf: recurse into the (memoized) unfolded stats of `(n, ls)`. Keep mathlib/core constants opaque in regime A; unfold them too (to axioms) in regime B if regime A is affordable.

3. Everything composes arithmetically under memoization: tree(c) = tree size with each project-const leaf replaced by the target's unfolded tree size (Float, as now); the unfolded DAG size of a theorem is the size of the union of the reachable per-constant node sets (computable by unioning hash sets of node keys, or by summing per-constant DAG sizes minus shared boundary terms — start with the union, it is exact). Reference counts aggregate the same way.

4. Emit, per theorem: `fulldag`, `log10fulltree`, `fullshared`, `fullmaxref`, plus `named_frac` = fraction of the full DAG contributed by subterms inside named-declaration boundaries vs. crossing them.

5. Analysis (extend `subterm_analysis.py`): redo the strata comparison and size-matching on full-unfolded quantities; the key statistic is whether the human-vs-machine gap in *diversity* of deep compression survives unfolding.

Cost. One ET rebuild (∼35 min; `lake exe cache get` then `lake build`), then one pass over the environment. Memory is the risk (global memo over 48k constants × instantiations); if it does not fit, process in topological order and retain only aggregate stats per `(Name, ls)`, dropping node sets once refcounts are folded in. Run the same smoke test as before: one `Generated` module + one human module before the full corpus.

4.2 P2: Subterm grain on the other corpora

PFR, FLT, Sphere (human + math-inc) have never been measured below the kernel grain. Now that disk is not scarce, rebuild each (∼30–60 min apiece, ∼8–10 GB each, *keep the builds*) and run `subterm_stats.lean` + `subterm_analysis.py` (the `stratum()` function needs a per-project rewrite — it is currently ET-specific). This turns §2.9 from an ET fact into a cross-corpus law (or not). Also the cheapest place to compare human vs. *AI-assisted* (math-inc Gauss) below naming.

4.3 P3: Queue (in rough order of value)

- **Dynamic EPT on git histories**: blueprints and Lean sources are in git; replaying the network's growth commit-by-commit tests whether projects *pass through* the transition (the 2022 paper's dynamics were static).
- **Mizar caveat**: the ATP premise selector was trained on human proofs, biasing toward human routes; rerun with an untrained selector or a different ATP corpus to bound the bias (it currently makes the observed 29% containment an *over*estimate of independent-search overlap, which strengthens the claim, but this should be stated with a measurement, not an argument).

- **Nexus 2026 corpus at kernel grain**: the 0%→97% naming-density shift (AlphaProof→Nexus) is currently textual-grain only.
- **Bistability battery on all corpora**: hysteresis/χ tests (transition tests B) ran on PFR only.
- **Paper**: fold P1/P2 outcomes into §4.4, then the draft is submittable as a workshop paper; the Simon-facing story is in `README.md`.

5 Infrastructure notes (changed assumptions)

- **Do not clear builds anymore.** Earlier rounds deleted `repos/*/.lake` after each extraction because the disk was nearly full. That constraint is gone (∼123 GB free as of this writing), and remote Ubuntu instances are available for bigger runs. Keep `.lake` directories; a warm build makes every new Lean-side question a minutes-scale iteration instead of an hour-scale one.
- **Rebuild recipe** (per project, from repo root):

  ```
  cd repos/<proj> && lake exe cache get && nohup lake build > build.log 2>&1 &
  ```

  Source repos are already on disk (`fetch_repos.sh` re-fetches all 11 if needed). Launch long builds detached with a done-marker file and poll; do not block a session on them.
- **Cloud recipe**: Ubuntu → install `elan` (which provides the pinned toolchain per `lean-toolchain`), `git clone` this repo, run `fetch_repos.sh`, then the recipe above; Python side needs only `numpy scipy networkx matplotlib python-louvain`.
- **Lean gotchas** (all hit in this project): private imports need `importAll := true` per module; theorem proof terms need `value? (allowOpaque := true)`; pre-module-system toolchains (4.28–4.31) need `extract_deps_legacy.lean`; the `public` modifier and the ET `equation` macro are textual-grain blind spots (kernel grain is ground truth); community detection on >5000 nodes must use Louvain, not `greedy_modularity`.
- **Model parameters** (for reproducing any Ising number): asymmetric Glauber, $\varepsilon = 1/(1 + e^{2\beta})$, prior field $p = 0.75$, 10 sweeps, seeds averaged where error bars are shown.

6 Pointers

- Repo: `https://github.com/scottviteri/ept-ai-analysis`
- Interactive report: `https://scottviteri.com/ept-ai-analysis/`
- Paper draft: `paper/main.pdf` ("Proofs after AlephZero", 11 pp)
- 2022 baseline: Viteri & DeDeo, *Epistemic phase transitions in mathematical proofs*, Cognition 2022 (its Coq networks are at the *subterm* grain — see `ept_appendix.pdf`)
- 2019 hash-consing original: `https://github.com/scottviteri/ManipulateProofTrees`